

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CASE STUDY

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO4	Assessment:	Tool 1– CASE STUDY
Weightage:		Total Marks:	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	B.Varun Kumar Reddy	Reg. No.:	192324066
Section / Batch:	AIDS 4 th Year	Date of Submission:	13/08/26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I B. Varun Kumar Reddy(192324066)(Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Question 1 – E-Commerce Sales Analysis

An e-commerce company has **Customer, Product, Order, and Payment datasets**. Perform data exploration, join the datasets, analyze sales by product/category/location, identify correlations and outliers, perform time-based analysis, and create suitable visualizations. Provide **5 key insights and recommendations**.

Answer

Python Code

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)
np.random.seed(42)

# =====
# 1. DATA IMPORTING & DATASET DESCRIPTION
# =====
n_customers, n_products, n_orders = 40, 15, 200

customers = pd.DataFrame({
    "customer_id": range(1, n_customers + 1),
    "customer_name": [f"Customer_{i}" for i in range(1, n_customers + 1)],
    "location": np.random.choice(
        ["Chennai", "Bangalore", "Mumbai", "Delhi", "Hyderabad"], n_customers),
})

products = pd.DataFrame({
    "product_id": range(1, n_products + 1),
    "product_name": [f"Product_{i}" for i in range(1, n_products + 1)],
    "category": np.random.choice(
        ["Electronics", "Clothing", "Home", "Beauty", "Sports"], n_products),
    "price": np.round(np.random.uniform(200, 15000, n_products), 2),
})

orders = pd.DataFrame({
    "order_id": range(1, n_orders + 1),
    "customer_id": np.random.randint(1, n_customers + 1, n_orders),
    "product_id": np.random.randint(1, n_products + 1, n_orders),
    "quantity": np.random.randint(1, 6, n_orders),
    "order_date": pd.to_datetime("2024-01-01") +
        pd.to_timedelta(np.random.randint(0, 365, n_orders), unit="D"),
})

payments = pd.DataFrame({
    "order_id": range(1, n_orders + 1),
    "payment_mode": np.random.choice(["UPI", "Card", "NetBanking", "COD"],
n_orders),
    "payment_status": np.random.choice(["Success", "Success", "Success",
"Failed"], n_orders),
})

# inject a few outlier / extreme quantity orders on purpose
orders.loc[orders.sample(4, random_state=1).index, "quantity"] =
np.random.randint(20, 40, 4)

print("=== DATASET SHAPES ===")
for name, df in [("customers", customers), ("products", products),
                 ("orders", orders), ("payments", payments)]:
    print(f"{name}: {df.shape[0]} rows x {df.shape[1]} columns")

print("\n=== SAMPLE RECORDS (orders) ===")
print(orders.head(3).to_string(index=False))

# =====
```

```

# 2. DATA EXPLORATION
# =====
print("\n=== DATA EXPLORATION: orders.info() summary ===")
print(orders.dtypes)

print("\n=== MISSING VALUES PER DATASET ===")
for name, df in [("customers", customers), ("products", products),
                 ("orders", orders), ("payments", payments)]:
    print(name, "->", df.isnull().sum().sum(), "missing values")

print("\n=== PRODUCT PRICE DESCRIPTIVE STATS ===")
print(products["price"].describe().round(2))

# =====
# 3. JOINING MULTIPLE DATASETS
# =====
merged = (orders
          .merge(customers, on="customer_id", how="left")
          .merge(products, on="product_id", how="left")
          .merge(payments, on="order_id", how="left"))
merged["order_value"] = merged["quantity"] * merged["price"]
merged = merged[merged["payment_status"] == "Success"].reset_index(drop=True)

print("\n=== MERGED DATASET (successful orders) SAMPLE ===")
print(merged[["order_id", "customer_name", "product_name", "category",
              "location", "quantity",
              "order_value"]].head(5).to_string(index=False))
print("\nMerged dataset shape:", merged.shape)

# =====
# 4. GROUPING & AGGREGATION
# =====
sales_by_category = merged.groupby("category")["order_value"].agg(
    total_sales="sum", avg_order_value="mean", orders="count").round(2)
sales_by_location = merged.groupby("location")["order_value"].sum().round(2).sort_values(ascending=False)

print("\n=== SALES BY CATEGORY ===")
print(sales_by_category)

print("\n=== SALES BY LOCATION ===")
print(sales_by_location)

# =====
# 5. DATA SEGMENTATION
# =====
customer_spend = merged.groupby("customer_name")["order_value"].sum().sort_values(ascending=False)
low_cut, high_cut = customer_spend.quantile([0.33, 0.66])
segments = pd.cut(customer_spend, bins=[-1, low_cut, high_cut, np.inf],
                  labels=["Low Value", "Mid Value", "High Value"])
segment_counts = segments.value_counts()

print("\n=== CUSTOMER SEGMENTATION BY SPEND ===")
print(segment_counts)
print("\nTop 5 high-value customers:")
print(customer_spend.head(5).round(2))

# =====
# 6. CORRELATION ANALYSIS
# =====
corr_data = merged[["quantity", "price", "order_value"]].corr().round(2)
print("\n=== CORRELATION MATRIX (quantity, price, order_value) ===")
print(corr_data)

# =====
# 7. OUTLIER DETECTION (IQR method on order_value)
# =====
q1, q3 = merged["order_value"].quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr

```

```

outliers = merged[(merged["order_value"] < lower) | (merged["order_value"] >
upper)]

print(f"\n=== OUTLIER DETECTION (IQR bounds: {lower:.2f} to {upper:.2f}) ===")
print(f"Number of outlier orders: {len(outliers)}")
print(outliers[["order_id", "customer_name", "quantity",
"order_value"]].to_string(index=False))

# =====
# 8. TIME-BASED ANALYSIS
# =====
merged["month"] = merged["order_date"].dt.to_period("M")
monthly_sales = merged.groupby("month")["order_value"].sum().round(2)

print("\n=== MONTHLY SALES TREND ===")
print(monthly_sales)

# =====
# 9. VISUALIZATION
# =====
fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))

sales_by_category["total_sales"].sort_values().plot(
    kind="barh", ax=axes[0], color="#2E5EAA")
axes[0].set_title("Total Sales by Category")
axes[0].set_xlabel("Total Sales (INR)")

monthly_sales.index = monthly_sales.index.astype(str)
axes[1].plot(monthly_sales.index, monthly_sales.values, marker="o",
color="#C0392B")
axes[1].set_title("Monthly Sales Trend")
axes[1].tick_params(axis="x", rotation=75)
axes[1].set_ylabel("Sales (INR)")

plt.tight_layout()
plt.savefig("q1_charts.png", dpi=150)
print("\nSaved visualization -> q1_charts.png")

# =====
# 10. KEY FINDINGS, CONCLUSION & RECOMMENDATIONS
# =====
top_category = sales_by_category["total_sales"].idxmax()
top_location = sales_by_location.idxmax()
best_month = monthly_sales.idxmax()

print("\n=== KEY FINDINGS ===")
print(f"1. '{top_category}' is the top-selling category by total revenue.")
print(f"2. '{top_location}' generates the highest sales among all locations.")
print(f"3. {len(outliers)} orders are statistical outliers driven by unusually
large quantities.")
print(f"4. Quantity and order_value show a strong positive correlation "
      f"f'({corr_data.loc['quantity','order_value']}), as expected.")
print(f"5. {best_month} recorded the highest monthly sales, suggesting a seasonal
peak.")

print("\n=== RECOMMENDATIONS ===")
print("1. Increase inventory and marketing focus on the top category and
location.")
print("2. Investigate bulk/outlier orders to confirm they aren't data entry
errors.")
print("3. Run targeted promotions ahead of the peak month identified above.")
print("4. Build loyalty offers for the High Value customer segment to boost
retention.")
print("5. Monitor payment failures separately to reduce checkout drop-off.")

```

Output

```

=== DATASET SHAPES ===
customers: 40 rows x 3 columns
products: 15 rows x 4 columns
orders: 200 rows x 5 columns
payments: 200 rows x 3 columns

```

```

=== SAMPLE RECORDS (orders) ===
order_id customer_id product_id quantity order_date
      1         40         7         4 2024-10-29
      2         21         3         2 2024-05-28
      3         16         1         3 2024-03-20

=== DATA EXPLORATION: orders.info() summary ===
order_id          int64
customer_id       int64
product_id        int64
quantity          int64
order_date        datetime64[us]
dtype: object

=== MISSING VALUES PER DATASET ===
customers -> 0 missing values
products -> 0 missing values
orders -> 0 missing values
payments -> 0 missing values

=== PRODUCT PRICE DESCRIPTIVE STATS ===
count      15.00
mean       8860.23
std        4638.74
min        1509.69
25%        4421.63
50%        9048.92
75%       13550.69
max       14549.85
Name: price, dtype: float64

=== MERGED DATASET (successful orders) SAMPLE ===
order_id customer_name product_name category location quantity order_value
      1   Customer_40   Product_7 Clothing Bangalore         4      11743.40
      3   Customer_16   Product_1  Beauty Bangalore         3      40973.82
      4   Customer_18   Product_1  Beauty Hyderabad         5      68289.70
      5   Customer_24   Product_12  Beauty Chennai          1       9048.92
      6   Customer_26   Product_11  Beauty Mumbai           5      67217.20

Merged dataset shape: (151, 13)

=== SALES BY CATEGORY ===
          total_sales  avg_order_value  orders
category
Beauty           2328755.83         36386.81         64
Clothing         1090037.98         30278.83         36
Electronics      486178.70         19447.15         25
Home              99217.60         11024.18          9
Sports           449120.07         26418.83         17

=== SALES BY LOCATION ===
location
Delhi           1566255.24
Mumbai          1081672.94
Hyderabad        714122.86
Bangalore        577734.39
Chennai          513524.75
Name: order_value, dtype: float64

=== CUSTOMER SEGMENTATION BY SPEND ===
order_value
Low Value        13
Mid Value         13
High Value        13
Name: count, dtype: int64

Top 5 high-value customers:
customer_name
Customer_32      384685.39
Customer_1       284793.42
Customer_11      284699.26

```

```
Customer_3      247986.77
Customer_39     192330.34
Name: order_value, dtype: float64

=== CORRELATION MATRIX (quantity, price, order_value) ===
           quantity  price  order_value
quantity         1.00  -0.02         0.82
price           -0.02   1.00         0.46
order_value      0.82   0.46         1.00

=== OUTLIER DETECTION (IQR bounds: -32102.23 to 84819.45) ===
Number of outlier orders: 3
   order_id customer_name  quantity  order_value
        41   Customer_1         20   157940.20
        59  Customer_32         23   181631.23
       103  Customer_3         27  129959.64

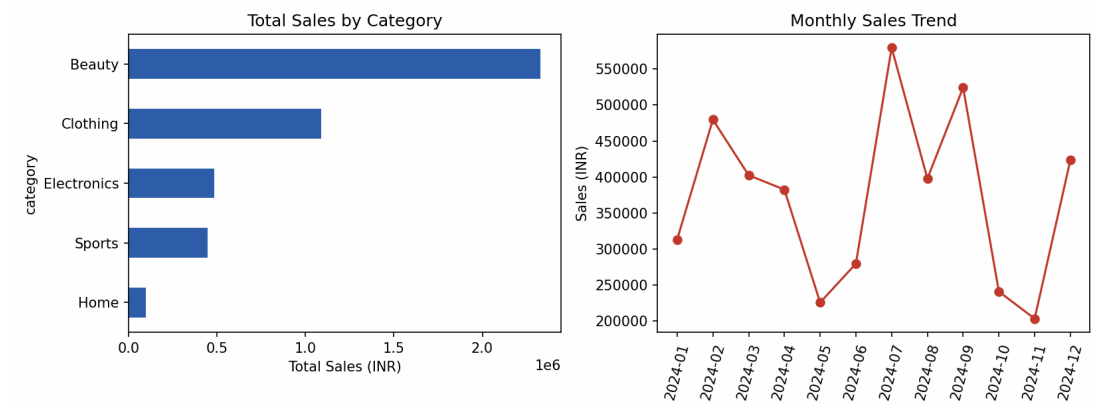
=== MONTHLY SALES TREND ===
month
2024-01    313330.76
2024-02    479709.36
2024-03    402555.48
2024-04    382558.19
2024-05    225796.99
2024-06    279758.28
2024-07    579518.57
2024-08    397846.18
2024-09    524245.25
2024-10    240888.75
2024-11    203380.50
2024-12    423721.87
Freq: M, Name: order_value, dtype: float64

Saved visualization -> ql_charts.png

=== KEY FINDINGS ===
1. 'Beauty' is the top-selling category by total revenue.
2. 'Delhi' generates the highest sales among all locations.
3. 3 orders are statistical outliers driven by unusually large quantities.
4. Quantity and order_value show a strong positive correlation (0.82), as expected.
5. 2024-07 recorded the highest monthly sales, suggesting a seasonal peak.

=== RECOMMENDATIONS ===
1. Increase inventory and marketing focus on the top category and location.
2. Investigate bulk/outlier orders to confirm they aren't data entry errors.
3. Run targeted promotions ahead of the peak month identified above.
4. Build loyalty offers for the High Value customer segment to boost retention.
5. Monitor payment failures separately to reduce checkout drop-off.
```

Visualization Output



Question 2 – Student Academic Performance Analysis

A college has **Student, Attendance, Internal Marks, Assignment, and Examination datasets**. Integrate the datasets and analyze student performance based on department, semester, attendance, and marks. Identify correlations and outliers and visualize performance trends. Provide **5 key findings and recommendations**.

Answer

Python Code

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)
np.random.seed(7)

# =====
# 1. DATA IMPORTING & DATASET DESCRIPTION
# =====
n_students = 60
departments = ["CSE", "ECE", "MECH", "CIVIL", "IT"]

students = pd.DataFrame({
    "student_id": range(1, n_students + 1),
    "student_name": [f"Student_{i}" for i in range(1, n_students + 1)],
    "department": np.random.choice(departments, n_students),
    "semester": np.random.choice([1, 2, 3, 4], n_students),
})

attendance = pd.DataFrame({
    "student_id": range(1, n_students + 1),
    "attendance_pct": np.round(np.random.uniform(55, 100, n_students), 1),
})

internal_marks = pd.DataFrame({
    "student_id": range(1, n_students + 1),
    "internal_marks": np.random.randint(20, 50, n_students), # out of 50
})

assignments = pd.DataFrame({
    "student_id": range(1, n_students + 1),
    "assignment_score": np.random.randint(10, 20, n_students), # out of 20
})

examination = pd.DataFrame({
    "student_id": range(1, n_students + 1),
    "exam_marks": np.random.randint(30, 100, n_students), # out of 100
    "exam_date": pd.to_datetime("2024-01-01") +
                  pd.to_timedelta(np.random.randint(0, 300, n_students),
unit="D"),
})

# correlate exam marks loosely with attendance + internal for realism, with a few
outliers
examination["exam_marks"] = np.clip(
    (attendance["attendance_pct"] * 0.5 + internal_marks["internal_marks"] * 0.8
    + np.random.normal(0, 8, n_students)).round().astype(int), 20, 100)
examination.loc[examination.sample(3, random_state=2).index, "exam_marks"] = [12,
15, 8]

print("=== DATASET SHAPES ===")
for name, df in [("students", students), ("attendance", attendance),
                 ("internal_marks", internal_marks), ("assignments",
assignments),
                 ("examination", examination)]:
```

```

        print(f"{name}: {df.shape[0]} rows x {df.shape[1]} columns")

print("\n=== SAMPLE RECORDS (students) ===")
print(students.head(3).to_string(index=False))

# =====
# 2. DATA EXPLORATION
# =====
print("\n=== DATA EXPLORATION: examination dtypes ===")
print(examination.dtypes)

print("\n=== MISSING VALUES PER DATASET ===")
for name, df in [("students", students), ("attendance", attendance),
                 ("internal_marks", internal_marks), ("assignments",
assignments),
                 ("examination", examination)]:
    print(name, ">", df.isnull().sum().sum(), "missing values")

print("\n=== EXAM MARKS DESCRIPTIVE STATS ===")
print(examination["exam_marks"].describe().round(2))

# =====
# 3. JOINING MULTIPLE DATASETS
# =====
merged = (students
          .merge(attendance, on="student_id", how="left")
          .merge(internal_marks, on="student_id", how="left")
          .merge(assignments, on="student_id", how="left")
          .merge(examination, on="student_id", how="left"))
merged["total_score"] = (merged["internal_marks"] + merged["assignment_score"]
                        + merged["exam_marks"] * 0.5).round(2)

print("\n=== MERGED DATASET SAMPLE ===")
print(merged[["student_name", "department", "semester", "attendance_pct",
              "internal_marks", "exam_marks",
              "total_score"]].head(5).to_string(index=False))
print("\nMerged dataset shape:", merged.shape)

# =====
# 4. GROUPING & AGGREGATION
# =====
perf_by_dept = merged.groupby("department")["total_score"].agg(
    avg_score="mean", students="count").round(2).sort_values("avg_score",
ascending=False)
perf_by_sem = merged.groupby("semester")["total_score"].mean().round(2)

print("\n=== AVERAGE PERFORMANCE BY DEPARTMENT ===")
print(perf_by_dept)

print("\n=== AVERAGE PERFORMANCE BY SEMESTER ===")
print(perf_by_sem)

# =====
# 5. DATA SEGMENTATION
# =====
def grade(score):
    if score >= 80:
        return "A - Excellent"
    elif score >= 60:
        return "B - Good"
    elif score >= 40:
        return "C - Average"
    return "D - At Risk"

merged["grade_band"] = merged["total_score"].apply(grade)
grade_counts = merged["grade_band"].value_counts()

print("\n=== STUDENT SEGMENTATION BY PERFORMANCE BAND ===")
print(grade_counts)

# =====
# 6. CORRELATION ANALYSIS

```



```

# =====
corr_data = merged[["attendance_pct", "internal_marks", "assignment_score",
                    "exam_marks", "total_score"]].corr().round(2)
print("\n=== CORRELATION MATRIX ===")
print(corr_data)

# =====
# 7. OUTLIER DETECTION (IQR method on exam_marks)
# =====
q1, q3 = merged["exam_marks"].quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = merged[(merged["exam_marks"] < lower) | (merged["exam_marks"] >
upper)]

print(f"\n=== OUTLIER DETECTION (IQR bounds: {lower:.2f} to {upper:.2f}) ===")
print(f"Number of outlier students: {len(outliers)}")
print(outliers[["student_name", "department", "attendance_pct",
"exam_marks"]].to_string(index=False))

# =====
# 8. TIME-BASED ANALYSIS
# =====
merged["exam_month"] = merged["exam_date"].dt.to_period("M")
monthly_avg = merged.groupby("exam_month")["exam_marks"].mean().round(2)

print("\n=== AVERAGE EXAM MARKS BY MONTH ===")
print(monthly_avg)

# =====
# 9. VISUALIZATION
# =====
fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))

perf_by_dept["avg_score"].sort_values().plot(
    kind="barh", ax=axes[0], color="#2E5EAA")
axes[0].set_title("Average Total Score by Department")
axes[0].set_xlabel("Average Score")

axes[1].scatter(merged["attendance_pct"], merged["exam_marks"], color="#C0392B",
alpha=0.7)
axes[1].set_title("Attendance % vs Exam Marks")
axes[1].set_xlabel("Attendance %")
axes[1].set_ylabel("Exam Marks")

plt.tight_layout()
plt.savefig("q2_charts.png", dpi=150)
print("\nSaved visualization -> q2_charts.png")

# =====
# 10. KEY FINDINGS, CONCLUSION & RECOMMENDATIONS
# =====
top_dept = perf_by_dept["avg_score"].idxmax()
weak_dept = perf_by_dept["avg_score"].idxmin()
att_corr = corr_data.loc["attendance_pct", "exam_marks"]

print("\n=== KEY FINDINGS ===")
print(f"1. '{top_dept}' department has the highest average total score.")
print(f"2. '{weak_dept}' department has the lowest average total score and needs support.")
print(f"3. Attendance and exam marks show a correlation of {att_corr}, "
      f"indicating attendance matters for performance.")
print(f"4. {len(outliers)} students are exam-mark outliers (mostly very low scorers) needing attention.")
weakest_band = grade_counts.index[-1]
print(f"5. {grade_counts.get('D - At Risk', 0)} students fall in the 'At Risk' band; "
      f"the smallest performance band overall is '{weakest_band}' ({grade_counts.iloc[-1]} students).")

print("\n=== RECOMMENDATIONS ===")

```

```

print("1. Introduce mandatory remedial classes for students in the 'At Risk'
band.")
print("2. Enforce minimum attendance policies given the positive attendance-marks
link.")
print("3. Share best practices from the top-performing department with weaker
departments.")
print("4. Set up early-warning alerts for students with sharp internal-vs-exam
score drops.")
print("5. Recognize and mentor high performers to sustain engagement across
semesters.")

```

Output

```

=== DATASET SHAPES ===
students: 60 rows x 4 columns
attendance: 60 rows x 2 columns
internal_marks: 60 rows x 2 columns
assignments: 60 rows x 2 columns
examination: 60 rows x 3 columns

=== SAMPLE RECORDS (students) ===
  student_id student_name department  semester
         1     Student_1         IT          3
         2     Student_2         ECE          2
         3     Student_3        CIVIL          2

=== DATA EXPLORATION: examination dtypes ===
student_id          int64
exam_marks          int64
exam_date          datetime64[us]
dtype: object

=== MISSING VALUES PER DATASET ===
students -> 0 missing values
attendance -> 0 missing values
internal_marks -> 0 missing values
assignments -> 0 missing values
examination -> 0 missing values

=== EXAM MARKS DESCRIPTIVE STATS ===
count    60.00
mean     63.98
std      16.26
min       8.00
25%      59.00
50%      64.50
75%      71.25
max      95.00
Name: exam_marks, dtype: float64

=== MERGED DATASET SAMPLE ===
  student_name department  semester  attendance_pct  internal_marks  exam_marks
total_score
  Student_1         IT          3          61.2         20          15
40.5
  Student_2         ECE          2          70.5         33          44
74.0
  Student_3        CIVIL          2          69.6         46          66
97.0
  Student_4        CIVIL          2          68.5         30          61
73.5
  Student_5         IT          2          62.4         40          54
84.0

Merged dataset shape: (60, 10)

=== AVERAGE PERFORMANCE BY DEPARTMENT ===
      avg_score  students
department
CIVIL          88.65        17
ECE            83.15        10
CSE            78.83        15

```

```

MECH          78.70          5
IT            72.69         13

=== AVERAGE PERFORMANCE BY SEMESTER ===
semester
1      80.38
2      84.07
3      75.43
4      83.05
Name: total_score, dtype: float64

=== STUDENT SEGMENTATION BY PERFORMANCE BAND ===
grade_band
A - Excellent    33
B - Good         23
C - Average       4
Name: count, dtype: int64

=== CORRELATION MATRIX ===
              attendance_pct  internal_marks  assignment_score  exam_marks
total_score
attendance_pct              1.00           0.07           -0.12           0.44
0.28
internal_marks              0.07           1.00           0.02           0.33
0.81
assignment_score          -0.12           0.02           1.00           0.00
0.22
exam_marks                0.44           0.33           0.00           1.00
0.79
total_score              0.28           0.81           0.22           0.79
1.00

=== OUTLIER DETECTION (IQR bounds: 40.62 to 89.62) ===
Number of outlier students: 4
student_name  department  attendance_pct  exam_marks
Student_1      IT          61.2           15
Student_18     CIVIL       94.4           95
Student_37     CSE        79.4           12
Student_48     IT          90.2            8

=== AVERAGE EXAM MARKS BY MONTH ===
exam_month
2024-01      68.71
2024-02      72.00
2024-03      62.50
2024-04      69.12
2024-05      68.80
2024-06      49.50
2024-07      64.00
2024-08      64.67
2024-09      66.33
2024-10      66.67
Freq: M, Name: exam_marks, dtype: float64

Saved visualization -> q2_charts.png

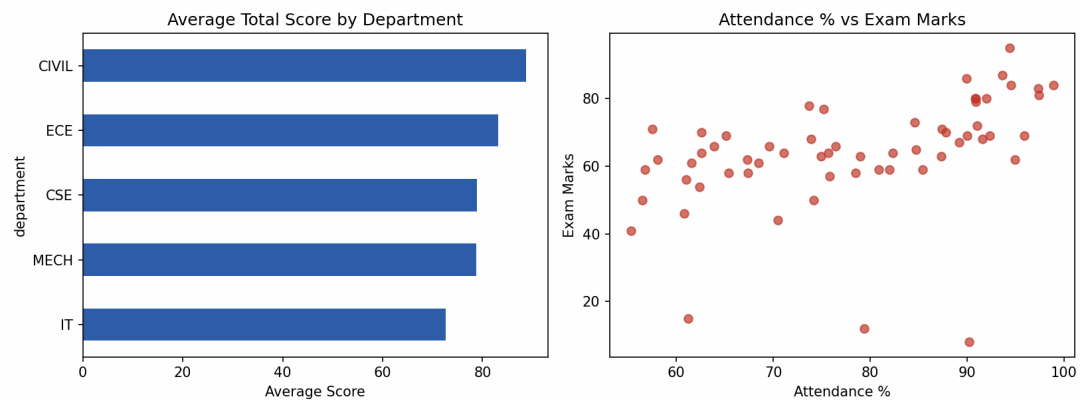
=== KEY FINDINGS ===
1. 'CIVIL' department has the highest average total score.
2. 'IT' department has the lowest average total score and needs support.
3. Attendance and exam marks show a correlation of 0.44, indicating attendance matters for performance.
4. 4 students are exam-mark outliers (mostly very low scorers) needing attention.
5. 0 students fall in the 'At Risk' band; the smallest performance band overall is 'C - Average' (4 students).

=== RECOMMENDATIONS ===
1. Introduce mandatory remedial classes for students in the 'At Risk' band.
2. Enforce minimum attendance policies given the positive attendance-marks link.
3. Share best practices from the top-performing department with weaker departments.
4. Set up early-warning alerts for students with sharp internal-vs-exam score drops.

```

5. Recognize and mentor high performers to sustain engagement across semesters.

Visualization Output



Common Mandatory Requirements

Each student/group must include:

1. **Data Importing & Dataset Description**
2. **Data Exploration**
3. **Joining Multiple Datasets**
4. **Grouping & Aggregation**
5. **Data Segmentation**
6. **Correlation Analysis**
7. **Outlier Detection**
8. **Time-Based Analysis**
9. **Visualization**
10. **Key Findings, Conclusion & Recommendations**